Building an Agent-Ready Infrastructure Developer Platform

Framings and source materials · October 2026

Treating the basics (docs, announcements, release updates) as part of the product rather than extras is the right instinct. On an infrastructure platform, a missed deprecation notice or an ambiguous doc can cause an outage. Below are framings to think with, then the source materials worth reading.

Framings

1. Platform as a product with two kinds of users: humans and agents

Platform engineering already treats the internal platform as a product: it has users, a roadmap, and adoption metrics. The new part is that a growing share of your users are agents acting for engineers: coding agents in IDEs, ops agents running remediation, and CI bots. For every capability, ask three questions:

- Can a person find and understand it?
- Can an agent find and call it safely?
- Can a person audit what the agent did? (In infrastructure, this matters most.)

2. One source of truth, rendered several ways

Avoid writing separate docs for people and for agents. Write structured content once (Markdown with front-matter, OpenAPI or JSON Schema for APIs) and generate the different outputs from it:

- A human docs site
- llms.txt and llms-full.txt for agent discovery
- AGENTS.md files in your repos
- An MCP server that exposes your platform's operations as tools

This is how you keep the two audiences from drifting apart.

3. The layered "agent-readable docs" stack

These standards each do a different job, so keep them separate:

- **llms.txt** is for discovery: it tells an agent which pages to read. Research notes it does not replace AGENTS.md or other agent files. Be realistic about what it buys you: a 2026 analysis found it doesn't measurably improve citation by AI search engines, but it is a real developer-experience win, since IDE agents increasingly look for it when pointed at a docs site. That second audience is exactly yours.
- **AGENTS.md** is the operating manual for agents working in a repo. It is now part of the Linux Foundation's Agentic AI Foundation, and the closest AGENTS.md in the directory tree takes precedence, a good fit for repos split across distributed teams, since each team owns the file for its own subtree.
- **MCP** is the live, callable layer. The 2026-07-28 revision is the current stable spec. It removed sessions entirely, so every call carries its own context and any request can land on any server instance, with no sticky routing or shared session stores. That suits an infrastructure platform you want to scale horizontally. The same revision hardened authorization to match real-world OAuth 2.0 and OpenID Connect deployments. Build to this revision from the start rather than to older tutorials.

4. For infrastructure, safety rules are part of the documentation

An agent that reads your docs will act on them. For every operation, document:

- Whether it can be undone
- How far its effects reach
- Whether a dry-run or plan mode exists
- What permissions it needs

Then put the same information into your API and MCP tool definitions as machine-readable annotations, for example `destructive: true` or `requires_approval`. A Terraform-style plan-then-apply flow is a good model for how both people and agents should use the platform.

5. Communications as three separate channels

These are different products for different readers:

- **Changelog:** complete, generated from commits, aimed at engineers and agents. Every entry follows the same structure so it can be parsed.
- **Announcements:** written for users, short, organized around "what can you do now that you couldn't before." Only for changes users will notice.
- **Release and deprecation notices across teams:** aimed at platform teams that depend on you. They need dates, what breaks, migration steps, and who to contact.

Many platforms fail by merging these into one noisy feed that nobody reads.

6. Calm, predictable releases across teams

For distributed teams, steady routine matters more than speed:

- Semantic versioning and a published deprecation policy (for example, at least 90 days for breaking changes)
- Machine-readable release events, so other teams' agents and CI can subscribe instead of watching Slack
- A single status and releases page that everything points to

7. Maturity is a map, not a goal

The CNCF maturity model's own authors warn that blindly pushing every area to the highest level is costly and sometimes harmful. Use it to pick what to improve next.

Source materials

Platform engineering foundations

- CNCF Platforms White Paper and the Platform Engineering Maturity Model. A refresh of both started in Q1 2026, so look for the newest version.
- *Team Topologies* (Skelton & Pais): the clearest way to think about platform teams versus the teams they serve, and how much complexity a team can carry.
- Google's *Site Reliability Engineering* and *SRE Workbook* (free online): especially the chapters on release engineering and managing change.

Docs for humans and agents

- Diátaxis: sorts docs into tutorials, how-to guides, reference, and explanation. Agents benefit from this split as much as people do.
- llmstxt.org: the spec. See platform.claude.com/llms.txt and docs.stripe.com/llms.txt as real examples.
- agents.md: the spec and examples.
- MCP specification 2026-07-28: read the changelog first.
- OpenAPI 3.1 and JSON Schema: the source your API reference and agent tools should both be generated from.
- *Docs for Developers* (Bhatti et al.): a practical book on writing technical docs.

Release communication

- Keep a Changelog, Semantic Versioning, and Conventional Commits: together these let you generate changelogs automatically.
- CloudEvents (a CNCF spec): for publishing release and deprecation events that other teams' tools can subscribe to.
- Stripe's API versioning and upgrade docs, and Kubernetes' deprecation policy: strong real-world examples of managing breaking changes at scale.

Developer portals

- Backstage (CNCF): study its software catalog and TechDocs, even if you build your own portal.

A practical first step

Before writing any platform code, write three short documents:

1. **A docs content model:** page types, required front-matter, and how pages get published to the docs site, llms.txt, and MCP.
2. **A change and deprecation policy.**
3. **A communications charter** that defines the three channels above, who writes each one, and who reads it.

These are cheap to write now and very hard to add later.

References

llms.txt Explained (May 2026), Codersera — https://codersera.com/blog/llms-txt-complete-guide-2026/

llms.txt: Discovery for AI Documentation, Emergent Mind — https://www.emergentmind.com/topics/llms-txt

Documentation stack for AI agents, Ravi Chaganti — https://ravichaganti.com/blog/documentation-stack-for-ai-agents/

MCP spec 2026-07-28 release, GitHub — https://github.com/modelcontextprotocol/specification/releases/tag/2026-07-28

MCP removes sessions, PPC Land — https://ppc.land/mcp-forces-ad-tech-to-rebuild-agent-servers-as-sessions-disappear/

Announcing the Platform Engineering Maturity Model, CNCF — https://tag-app-delivery.cncf.io/blog/announcing-the-platform-engineering-maturity-model/

2026 refresh of the whitepaper and maturity model, CNPE — https://cnpe.netlify.app/blog/2026-initiatives-announcement

agents.md — https://agents.md/